

COMPLETE LEARNING ROADMAP

TypeScript

From Zero to Mastery

*From JavaScript origins to advanced type-level programming***6**

PHASES

40+

TOPICS

18

PROJECTS

~6 mo

TIMELINE

PHASE 0 The Story Before TypeScript*You can't appreciate TS without understanding what JS was*

JavaScript: The Origin Story

To understand why TypeScript exists, you first need to understand the history and shortcomings of JavaScript — the language it extends.

A Timeline of JavaScript's Evolution

Year	What Happened
1995 — Birth	Brendan Eich creates JS in 10 days at Netscape. Originally called Mocha, meant for browser form validation.
1997 — ECMAScript 1	JavaScript gets formally standardized by ECMA International — hence why versions are called ES5, ES6, ESNext.
2009 — Node.js	Ryan Dahl releases Node.js. JS escapes the browser and can now run on servers, read files, and power APIs.
2012 — TypeScript born	Microsoft releases TypeScript, led by Anders Hejlsberg (creator of C#). TS compiles down to plain JavaScript.

2015 — ES6 (ES2015)	JavaScript's biggest upgrade: class, let/const, arrow functions, destructuring, modules. JS finally felt like a real language.
2020s — TS standard	TypeScript becomes the industry standard, used by Google, Microsoft, Airbnb, Slack, and most major open-source projects.

Why JavaScript Needed TypeScript

Silent Runtime Bugs JS lets you call undefined() and it only crashes at runtime. TS catches this before you even run the code.	Dynamic Typing A variable can change from a number to a string to an object. Great for quick scripts, terrible for large codebases.
No Contracts Functions have no declared signatures — you never know what a function expects as input or what it returns.	Poor Tooling IDEs couldn't autocomplete or warn about errors because everything was a mystery until the code actually ran.

Key Concept: TypeScript is a superset of JavaScript. Every valid JavaScript file is valid TypeScript. TS only adds things on top — most importantly, a static type system that lives at compile time and gets erased before the code runs.

PHASE 1 JavaScript Fundamentals

You must know JS deeply before you can appreciate what TS adds

Don't skip this phase. TypeScript is meaningless without a solid foundation in JavaScript. You need to understand JS's quirks — its type coercion, prototype chain, closures, and async model — because TypeScript is designed to solve those specific problems.

Core Topics — Weeks 1 to 3

Variables & Scoping var vs let vs const, hoisting, block scope, temporal dead zone.	Data Types string, number, boolean, null, undefined, Symbol, BigInt — and the difference between primitives and objects.
Type Coercion JS silently converts types. Why "5" + 1 = "51" but "5" - 1 = 4. Essential for understanding why TS exists.	Functions Declarations vs expressions, arrow functions, default params, rest/spread, first-class functions.
Objects & Arrays	Prototypes & this

Reference vs value types, destructuring, spread operator, built-in array/object methods.

Closures

Functions that remember their outer scope — one of JavaScript's most powerful and misunderstood features.

ES6+ Modern Syntax

Classes, modules (import/export), template literals, optional chaining, nullish coalescing.

How JS inheritance really works. Why this is confusing and context-dependent. The prototype chain.

Async JavaScript

Callbacks → Promises → async/await. The event loop. How JS handles waiting without blocking.

Error Handling

try/catch/finally, custom Error objects, error propagation patterns.

Phase 1 Projects



JS Quiz Engine

A CLI quiz app. Store questions as plain objects, loop through them, track score, handle user input. No libraries — pure vanilla JS only.

Concepts: [objects](#) · [arrays](#) · [loops](#) · [functions](#) · [closures](#)



Async Weather Fetcher

Fetch weather from a public API using fetch() and async/await. Handle loading, success, and error states. Practice the full async lifecycle.

Concepts: [promises](#) · [async/await](#) · [error handling](#) · [JSON parsing](#)

PHASE 2 TypeScript Fundamentals

What TypeScript actually adds on top of JavaScript

The Type System — Weeks 4 to 6

TypeScript's entire purpose is to add a static type system to JavaScript. This phase covers the core of that system — how you describe what types your variables, functions, and data structures hold.

Type Annotations

Explicitly telling TS a variable's type: `let name: string = "Alice"`. The basic syntax.

Primitive Types

string, number, boolean, null, undefined, void, never, unknown, any — and when to use each.

Enums

Named constants: `enum Direction { Up, Down, Left, Right }`. Numeric and string enums.

Type Inference

TS is smart — it infers types automatically from assignments. You don't always need to annotate.

Arrays & Tuples

`string[]`, `Array<number>`; tuples are fixed-length typed arrays: `[string, number]`.

Union Types

A variable that can be one of several types: `string | number | null`.

Intersection Types

Combining types: TypeA & TypeB — an object must satisfy both shapes simultaneously.

Interfaces

Define object "shapes": interface User { name: string; age: number }. The most common TS construct.

Type Aliases

Create reusable named types: type UserID = string | number.

Functions in TS

Typed parameters and return values, optional params (name?: string), function overloads.

Critical Insight: TypeScript's type system lives entirely at compile time. Once your code runs in the browser or Node.js, all type information is erased. TypeScript compiles to plain JavaScript. Types are a developer tool, not a runtime feature — this is why runtime validation libraries like Zod exist.

Phase 2 Projects

**Typed Todo List (CLI)**

Rewrite your JS quiz engine or build a todo app fully typed. Define an interface Todo, write typed functions, use typed arrays. Observe how TS catches bugs instantly as you code.

Concepts: [interfaces](#) · [type aliases](#) · [arrays](#) · [unions](#) · [enums](#)

**Type-Safe Config Parser**

Parse a JSON config file into a strongly-typed TS object. Validate every field's type. Throw descriptive errors on mismatches. A perfect exercise in null-safety.

Concepts: [interfaces](#) · [type narrowing](#) · [null checks](#) · [error handling](#)

PHASE 3 Intermediate TypeScript

Where TS gets truly powerful — generics and advanced types

Generics — The Most Important Concept

Generics are the single most important advanced concept in TypeScript. They allow you to write code that is both reusable and type-safe — code that works for any type without sacrificing type checking. Master generics and you have mastered the heart of TypeScript.

Timeline: Weeks 7 to 9

Generic Functions

function identity<T>(arg: T): T — write once, works for any type passed in.

Generic Interfaces

interface Response<T> { data: T; status: number } — parameterize your structures.

Generic Constraints**Type Guards**

`<T extends object>` — restrict what types a generic will accept using `extends`.

Literal Types

`type Direction = "north" | "south" | "east" | "west"` — restrict to exact values.

Mapped Types

Transform existing types: make all properties optional, readonly, nullable, etc.

Utility Types

Built-in: `Partial<T>`, `Required<T>`, `Readonly<T>`, `Pick<T, K>`, `Omit<T, K>`, `Record<K, V>`.

`typeof`, `instanceof`, and custom guards (`x is string`) — safely narrow union types.

Template Literal Types

`type EventName = `on${string}`` — build types from string pattern combinations.

Conditional Types

`T extends string ? "yes" : "no"` — if/else logic at the type level.

keyof & typeof

Extract keys of a type as a union; grab the type of a runtime value statically.

Phase 3 Projects



Generic In-Memory Database

A typed data store that can hold any model: `DB<User>`, `DB<Product>`. Supports full CRUD operations, all fully type-safe. This project forces you to truly master generics.

Concepts: [generics](#) · [constraints](#) · [mapped types](#) · [utility types](#)



Type-Safe API Client

A wrapper around `fetch()` using generics: `get<User>("/api/user")` returns a properly typed response. Handle loading and error states with discriminated unions.

Concepts: [generics](#) · [discriminated unions](#) · [conditional types](#) · [async/await](#)



State Machine Library

Model state transitions (traffic lights, order status) using literal types and mapped types to ensure only valid state transitions are allowed — enforced at compile time.

Concepts: [literal types](#) · [mapped types](#) · [template literals](#) · [keyof](#)

PHASE 4 Object-Oriented TypeScript

Classes, access modifiers, and design patterns

Classes & OOP — Weeks 10 to 12

JavaScript has classes, but TypeScript supercharges them with access modifiers, abstract classes, and full interface implementation. This phase shows you how to design clean, maintainable object-oriented systems with TypeScript.

Classes & Constructors

Access Modifiers

Full class syntax, typed properties, constructor parameter shorthand with access modifiers.

Abstract Classes

Classes that define a blueprint but cannot be instantiated directly.

Inheritance

extends, method overriding, super() calls, polymorphism patterns.

Design Patterns

Singleton, Factory, Observer, Builder — implement classic patterns with full type safety.

public, private, protected — control visibility and encapsulation precisely.

Implementing Interfaces

class Dog implements Animal — use interfaces as contracts that classes must fulfill.

Decorators

Metadata annotations: @Component, @Injectable — how frameworks like Angular use TS.

SOLID Principles

How TS interfaces and abstract classes enable clean, maintainable OOP architecture.

Phase 4 Projects



Bank Account System

Model Account, SavingsAccount, and CheckingAccount with inheritance. Use private balances, abstract methods, and interface contracts for transaction validation.

Concepts: [classes](#) · [inheritance](#) · [abstract](#) · [interfaces](#) · [access modifiers](#)



Plugin System (Observer Pattern)

Build an event emitter with fully typed events. Each plugin implements a common interface. Demonstrates OOP combined with generics elegantly.

Concepts: [observer pattern](#) · [generics](#) · [interfaces](#) · [decorators](#)

PHASE 5 Advanced TypeScript

Type-level programming — using types like a language within a language

Advanced Type Patterns — Weeks 13 to 18

This is where TypeScript's type system becomes truly extraordinary. You can write logic — conditionals, loops, extraction — entirely at the type level. This phase separates TypeScript experts from everyone else.

infer keyword

Extract types from within other types. Example: type ReturnType<T> = T extends () => infer R ? R : never.

Discriminated Unions

Use a common literal tag field so TS can narrow between union members precisely and exhaustively.

Recursive Types

Types that reference themselves — model trees, deeply nested JSON, infinite data structures.

Declaration Merging

Extend existing interfaces or modules across files — TypeScript's additive type system.

Module Augmentation Add type declarations to third-party libraries that lack complete coverage.	Variance & Covariance How types behave when assigned to supertypes/subtypes — structural typing explained deeply.
Exhaustive Narrowing Never-based exhaustiveness checks in switch statements — ensure all cases are handled.	tsconfig Deep Dive strict, noImplicitAny, strictNullChecks, target, lib, paths, project references.
Writing .d.ts Files Type declaration files — how to add TypeScript types to plain JavaScript libraries.	satisfies Operator The satisfies keyword (TS 4.9) — validate a type without losing precision through widening.

Phase 5 Projects

☐	Build Your Own Utility Types Library Re-implement Partial, Required, Pick, Omit, ReturnType, Parameters from scratch using keyof, infer, and mapped types. This forces you to truly understand TS internals. <i>Concepts: infer · mapped types · conditional types · keyof · recursive types</i>
☐	Type-Safe Form Builder A schema-driven form system where field types, validation rules, and output types are all inferred from a single schema definition. No runtime type errors possible. <i>Concepts: discriminated unions · infer · generics · template literals</i>
☐	SQL Query Builder (Type-Level) Model a DSL where select("name", "age").from(users) returns a type that only contains name and age. The return type changes dynamically based on what columns you query. <i>Concepts: template literals · conditional types · recursive types · infer</i>

The Single Best Exercise: Re-implement the built-in utility types from scratch. If you can write Partial<T>, DeepReadonly<T>, and ReturnType<T> yourself without looking them up, you truly understand TypeScript's type system.

PHASE 6 Ecosystem, Tooling & Real-World TS

How TypeScript fits into the broader JavaScript ecosystem

Core Tooling

Tool	Purpose
------	---------

tsc	The TypeScript compiler. Runs type checking and compiles .ts files to .js.
npm / pnpm / yarn	Package managers. How you install TypeScript, type definitions, and all dependencies.
esbuild / tsx	Fast TypeScript execution without a separate compile step.
ESLint + typescript-eslint	Code linting for TypeScript — catches style issues and potential bugs beyond type errors.
Prettier	Code formatter — enforces consistent style across your entire codebase.
Vitest / Jest	Unit testing frameworks with TypeScript support. Vitest is the modern choice.
Zod	Runtime validation library. Since types are erased at runtime, Zod validates data from APIs and user input.
@types/* (DefinitelyTyped)	Community-maintained type definitions for JavaScript libraries that don't ship their own types.

Key Concepts

TS + Node.js Running TypeScript on the server: ts-node, tsx, Node.js type definitions (@types/node).	Zod (Runtime Validation) TS types are erased at runtime. Zod validates data AND infers TS types from schemas — the best of both worlds.
Testing Typed Code Unit testing with Vitest, mocking typed modules, testing type correctness with expect-type.	Monorepos Managing multiple TS packages with shared types using project references and path aliases.

Capstone Projects

☐	Publish a Typed NPM Library Write a small utility library (date formatter, validation toolkit) in TypeScript and publish it to npm with proper .d.ts type exports. Others can install it with full type safety. <i>Concepts: tsconfig · .d.ts files · npm publish · generics · documentation</i>
☐	REST API with Full Type Safety Build a Node.js API where request bodies, query params, and responses are all validated with Zod and typed end-to-end. Zero implicit any in the entire codebase. <i>Concepts: zod · strict tsconfig · generics · error handling · testing</i>

**Type-Safe Event Bus**

An event system where each event name maps to a specific payload type. Emitting or listening with the wrong data type is a compile error. Showcases advanced generics and mapped types together.

Concepts: [advanced generics](#) · [mapped types](#) · [template literals](#) · [infer](#)

Best Learning Resources

☐ **Official TS Handbook**

typescriptlang.org/docs — the canonical reference. Very readable and well organized.

☐ **Type Challenges**

github.com/type-challenges — 100+ TS puzzles from easy to extreme. Do these regularly.

☐ **TypeScript Playground**

typescriptlang.org/play — test any TypeScript concept instantly in the browser. No setup.

☐ **Total TypeScript**

Matt Pocock's free workshops on advanced TypeScript. Probably the best advanced resource available.

☐ **Effective TypeScript (Book)**

Dan Vanderkam. 62 specific, practical ways to improve your TypeScript. Excellent for depth.

☐ **Fireship (YouTube)**

Fast, visual TypeScript explainers. Great for conceptual overviews before going deep.

Remember: TypeScript is a tool for thinking about your code, not just a syntax requirement. The goal isn't to annotate everything — it's to design systems where incorrect usage is impossible to express. When you reach that point, you understand TypeScript.

TypeScript Learning Roadmap · 6 Phases · ~6 Months